

Paging

Android Architecture 역량강화

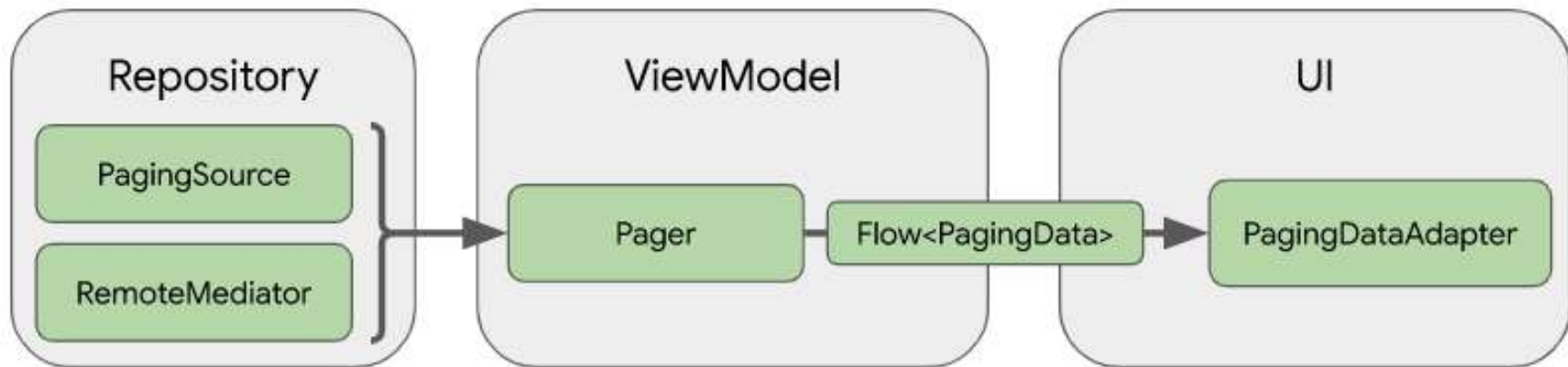
Paging

- RecyclerView 에서 페이징 처리를 쉽고 효율적으로 작성하게 하기 위한 라이브러리
- 페이징 된 데이터의 메모리 내 캐싱
- 요청 중복 제거 기능이 기본으로 제공
- 데이터의 끝까지 스크롤할 때 어댑터가 자동으로 데이터를 요청
- Flow, LiveData, RxJava 를 지원

```
implementation("androidx.paging:paging-runtime-ktx:$paging_version")
```

Paging

- Repository Layer, ViewModel Layer, UI Layer 로 구성





Paging



- PagingSource : 네트워크 또는 데이터베이스에서 페이징 데이터 로드 담당
- RemoteMediator : 데이터베이스 및 네트워크에서 페이징 데이터 로드 담당
- Pager : 등록된 DataSource 를 이용해 데이터를 Flow, LiveData, RxJava 로 발행
- PagingData : 페이징 된 데이터
- PagingConfig : 페이징 설정
- PagingAdapter : PagingData 로 항목을 구성하는 RecyclerView Adapter



02 PagingSource

PagingSource

- 네트워크 또는 데이터베이스에서 페이징 데이터를 로드 하는 역할
- PagingSource 에 의해 로딩된 데이터는 PagingData 객체로 관리
- PagingSource 를 상속받아 작성

```
class UserDataSource(private val dao: UserDAO) : PagingSource<Int, User>() {  
}
```

- 페이징을 위한 데이터를 획득하는 역할자.
- 캐싱이 없는 경우는 PagingSource 만 이용

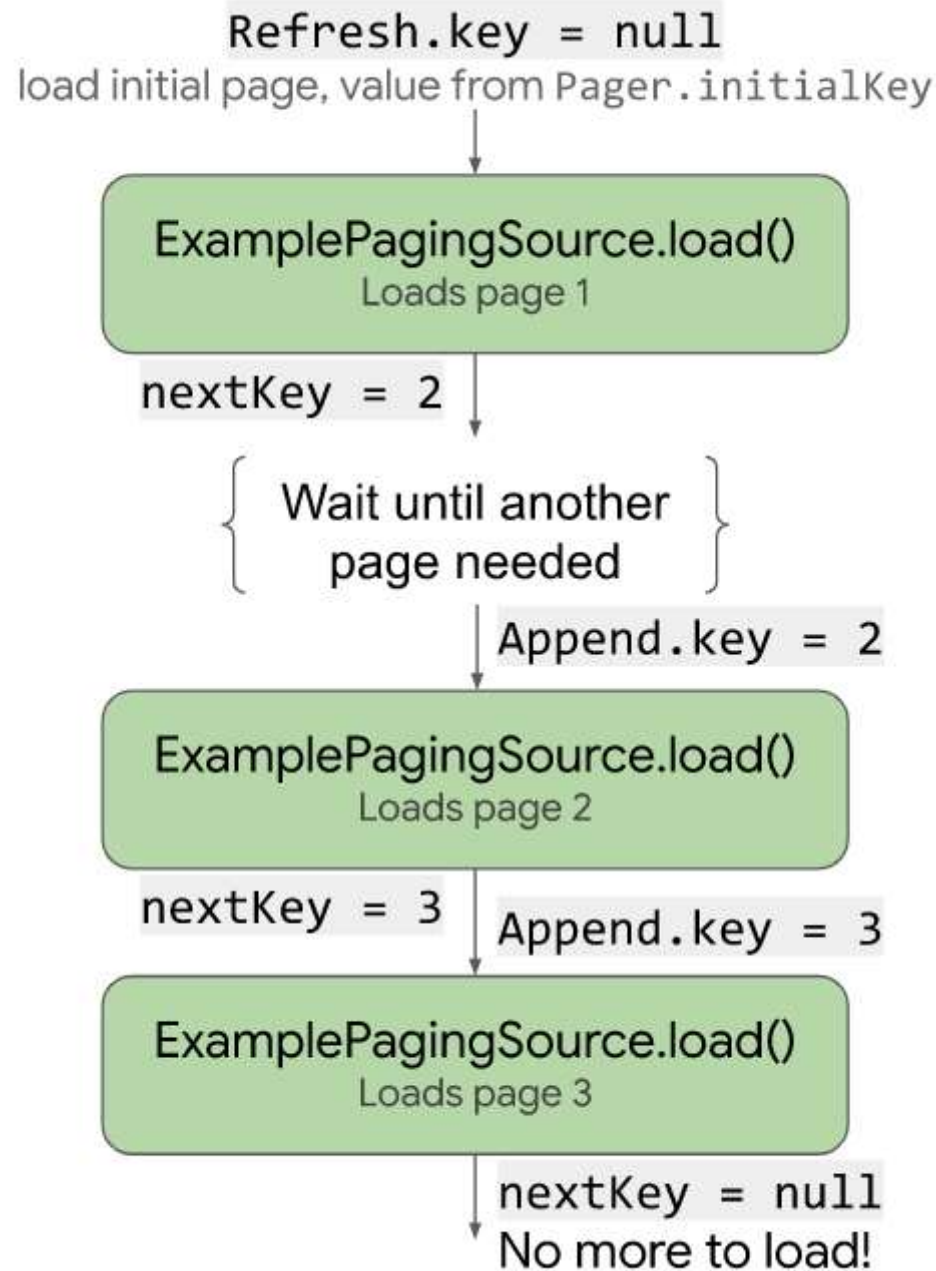
load()

- 필수 override 함수, 데이터 로드 정의

```
override suspend fun load(params: LoadParams<Int>): LoadResult<Int, User> { }
```

- LoadParam : 데이터를 로드하기 위한 정보
 - param.key : 로드 할 키
 - param.loadSize : 로드 할 항목 수
- LoadResult : 데이터 로드 결과 타입
 - LoadResult.Page : 로드 성공한 경우이며 Page 에 로드 데이터 포함
 - LoadResult.Error : 로드 실패한 경우
- LoadResult.Page
 - data : 로드된 데이터
 - preKey : 이전 페이지 key
 - nextKey : 이후 페이지 key

load()



load()

```
override suspend fun load(params: LoadParams<Int>): LoadResult<Int, User> {  
    try {  
        val page = params.key ?: 1  
        val end = start + params.loadSize  
  
        .....  
  
        return LoadResult.Page(  
            data = response,  
            prevKey = prevKey,  
            nextKey = page.plus(1)  
        )  
    } catch (e: Exception) {  
        e.printStackTrace()  
        return LoadResult.Error(e)  
    }  
}
```

Pager

- Paging 의 Entry Point
- PagingSource 와 PagingConfig 등록
- 등록된 PagingSource 를 PagingConfig 정보로 이용해 데이터를 Flow, LiveData, RxJava 로 발행
- PagingConfig
 - pageSize : DataSource 에 의해 한번에 로딩될 데이터 개수
 - maxSize : DataSource 에 의해 로딩될 데이터의 최대 개수
 - initialLoadSize : 초기 로딩될 데이터 개수

```
Pager(  
    config = PagingConfig(  
        pageSize = 10,  
        maxSize = 100,  
        initialLoadSize = 10  
    )  
) {  
    UserDataSource(dao)  
}.flow.cachedIn(viewModelScope)
```

PagingDataAdapter

- RecyclerView Adapter
- PagingDataAdapter 를 상속받아 작성
- DiffUtil.ItemCallback 필수 지정

```
class UserAdapter: PagingDataAdapter<User, ViewHolder>(diffCallback) {  
    .....  
    companion object {  
        val diffCallback = object : DiffUtil.ItemCallback<User>() {  
            override fun areItemsTheSame(oldItem: User, newItem: User): Boolean {  
                //수정....  
                return oldItem.email == newItem.email  
            }  
            override fun areContentsTheSame(oldItem: User, newItem: User): Boolean {  
                return oldItem == newItem  
            }  
        }  
    }  
}
```

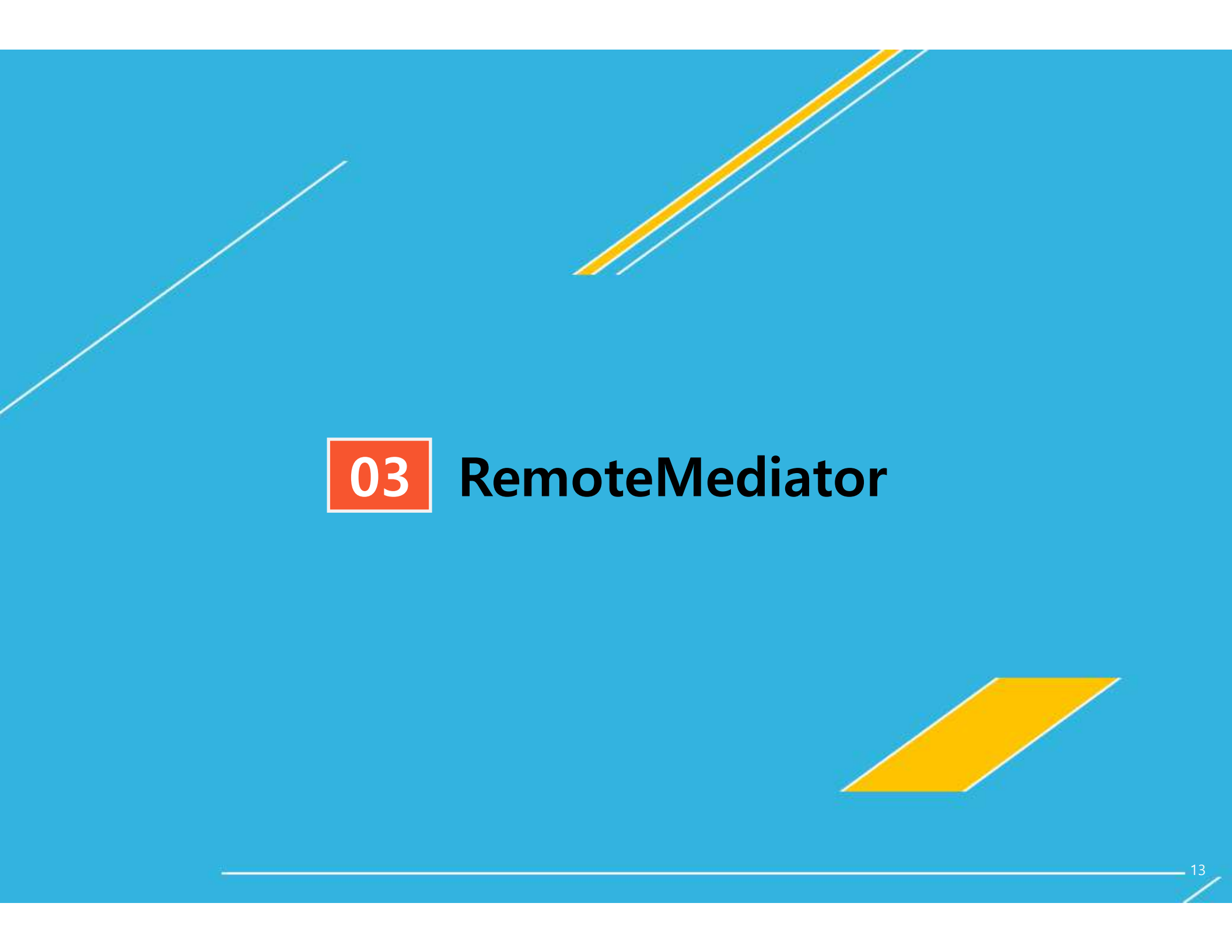


submitData()



- Pager 의 PagingSource 가 발행한 데이터를 Adapter 에 연결
- Adapter 의 submitData() 함수 이용

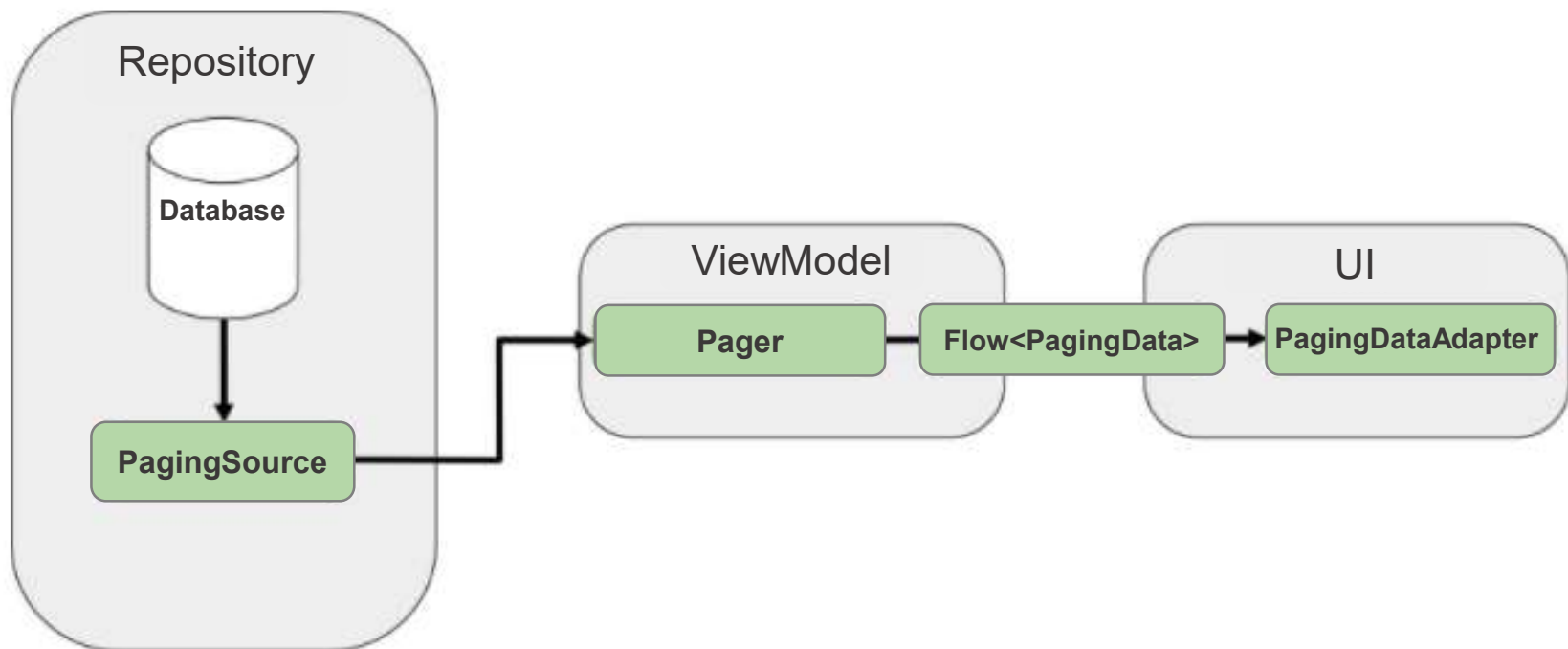
```
viewModel.allUsers.collect {  
    adapter.submitData(it)  
}
```



03 RemoteMediator

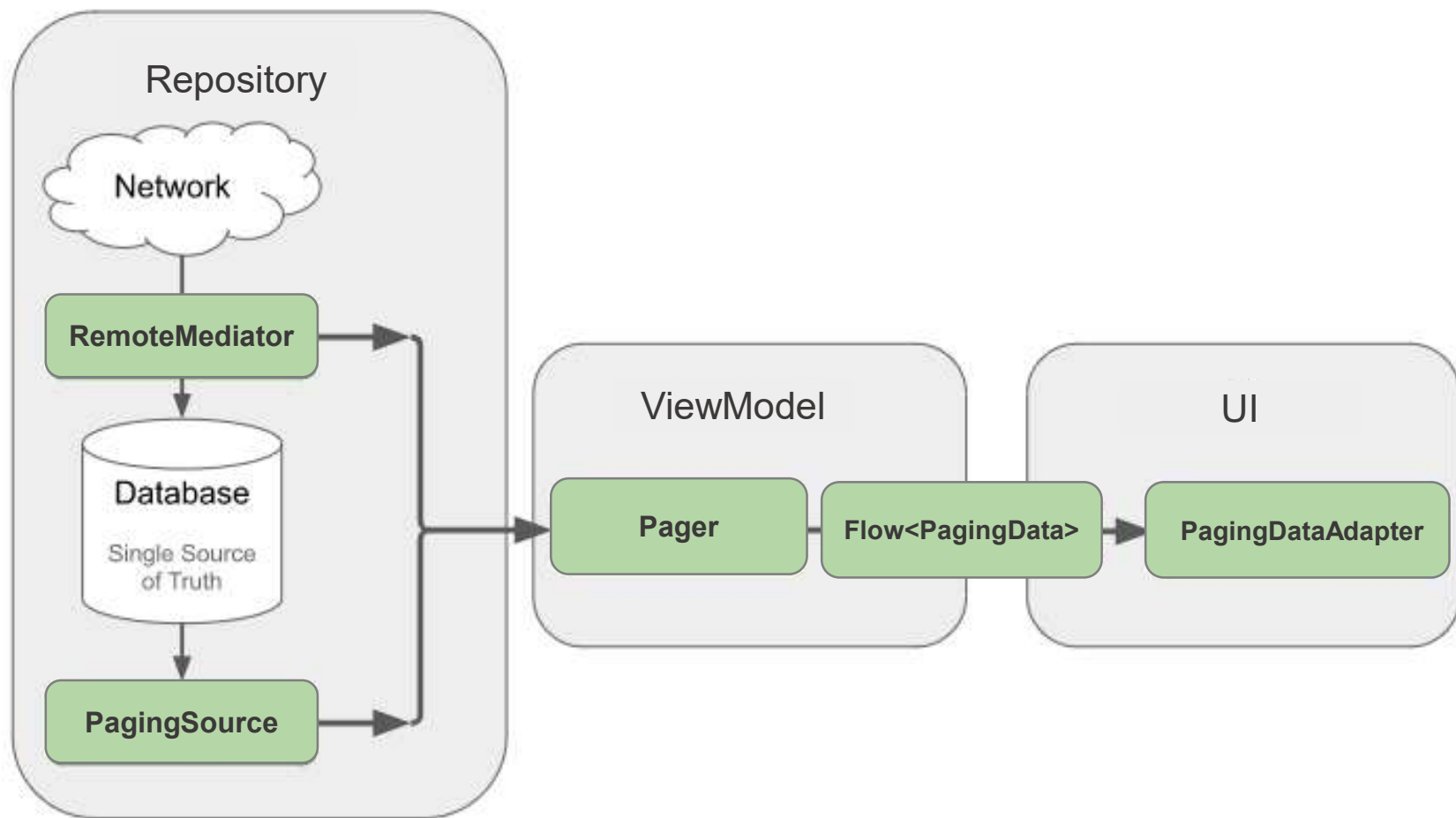
RemoteMediator

- ExperimentalPagingApi
- 페이징 데이터를 로딩하는 역할
- 데이터 캐싱이 필요한 경우 추가
- RemoteMediator 를 사용하지 않는 경우의 구조



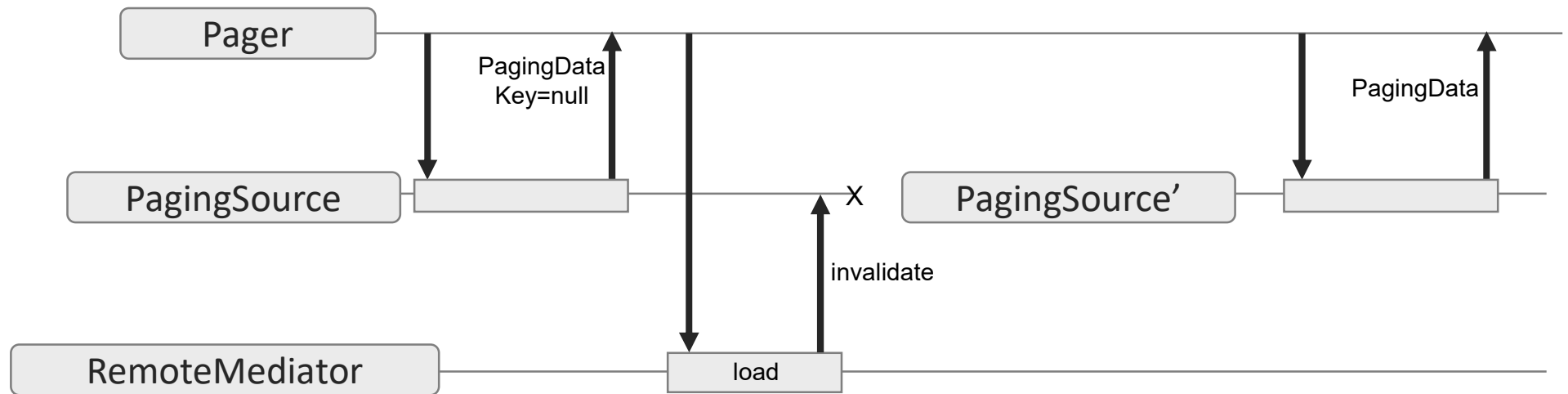
RemoteMediator

- RemoteMediator 를 사용하는 경우의 구조.



RemoteMediator

- RemoteMediator 의 흐름



RemoteMediator

- RemoteMediator 상속으로 작성
- PagingSource 와 동일 타입으로 제네릭 선언

```
@OptIn(ExperimentalPagingApi::class)
class UserRemoteMediator(
    private val db: AppDatabase,
    private val redditApi: RetrofitService
) : RemoteMediator<Int, User>() {
}
```

- Pager 에 RemoteMediator 등록

```
Pager(
    config = PagingConfig(
        pageSize = 10,
        maxSize = 100,
        initialLoadSize = 10
    ),
    remoteMediator = UserRemoteMediator(db, retrofitService)
) {
    db.userDAO().getUsers()
}
```

RemoteMediator

```
@OptIn(ExperimentalPagingApi::class)
class UserRemoteMediator(private val db: AppDatabase, private val redditApi: RetrofitService
) : RemoteMediator<Int, User>() {
    private val postDao = db.userDAO()
    private val remoteKeyDao = db.getKeysDao()
    override suspend fun initialize(): RemoteMediator.InitializeAction {
        return RemoteMediator.InitializeAction.LAUNCH_INITIAL_REFRESH
    }
    override suspend fun load( loadType: LoadType, state: PagingState<Int, User>): MediatorResult {
        try {
            var loadKey = when (loadType) {
                .....
            }
            val page: Int = loadKey?.let { loadKey + 1 } ?: 1
            val data: List<User> = redditApi.getServerData(.....).body()?.results ?: listOf<User>()

            db.withTransaction {
                .....
            }
            return MediatorResult.Success(endOfPaginationReached = data.isEmpty())
        } catch (e: Exception) {
            return MediatorResult.Error(e)
        }
    }
}
```

Compose

```
@Composable
fun MyScreen(viewModel: MyViewModel) {
    val items = viewModel.pagingFlow.collectAsLazyPagingItems()

    LazyColumn {
        items(items) { item ->
            item?.let { MyItemCard(it) }
        }

        // 로딩 상태 처리
        when (items.loadState.append) {
            is LoadState.Loading -> {
                item { CircularProgressIndicator() }
            }
            is LoadState.Error -> {
                item { Text("에러 발생") }
            }
            else -> {}
        }
    }
}
```